

Lab 2 — The Paradigm Shift: FreeRTOS Tasks and Drift-Free Timing

ESP32-S3 FIRMWARE LAB SERIES · PHASE 1 · FOUNDATIONS

Estimated time	2–3 hrs
Introduces	tasks, priorities, blocking, <code>vTaskDelayUntil</code>
Prerequisites	Labs 0–1.
Beyond the board	Nothing.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§2.1 objective**, **§2.2 parts and wiring**, **§2.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

2.1 Objective & TM4C bridge

Run two independent pieces of code at different rates without either one interfering with the other, and learn the difference between a delay that drifts and one that does not. This is the most important conceptual lab in Phase 1; everything from Lab 7 onward assumes it.

TM4C BRIDGE

The big one. On the TM4C, `main()` was a super-loop and `delayMs()` was a busy-wait that burned the CPU counting nops. Any bare-metal interrupt chapter already makes the case against spinning: an infinite loop wastes cycles and stops scaling once several peripherals are attached, and interrupts were your escape hatch. What you never had was a **scheduler**. FreeRTOS is already running before your code starts, `app_main` is itself a task, and `vTaskDelay` hands the CPU to other tasks and comes back. Busy-waiting is now an actual bug that starves the scheduler. This is the single biggest mental change from bare metal.

2.2 Parts & wiring

Item	Status
ESP32-S3 board + USB-C cable	nothing external

No wiring. All the output is serial log text, so the monitor is your only instrument. The next lab that needs hardware is Lab 3, which wants a single jumper wire.

2.3 How it works

"But isn't it just a framework?"

A fair question, and answering it precisely sharpens the whole idea. Your TM4C super-loop was **cooperative and manual**: the loop ran top to bottom forever, and the only thing that could interrupt it was a hardware ISR doing the bare minimum before returning. You scheduled by hand-ordering the if-checks, and if one section took 200 ms, everything behind it waited, because nothing could yank the CPU away.

FreeRTOS is **preemptive and automatic**: a periodic timer interrupt (the tick, every 10 ms by default) can forcibly pause a running task mid-line and switch to a higher-priority task that just became ready. The paused task did not finish and did not volunteer — it was preempted. So yes, it is a framework, but the specific thing it gives you is a multitasking scheduler for your ordinary, non-interrupt code: many concurrent flows that the kernel time-slices and prioritises, plus **blocking primitives** that let a task wait without burning cycles, plus ISR-safe queues, semaphores and mutexes. The ISRs are still ISRs, same hardware, same job as on the TM4C.

The four states a task can be in

Every task is in exactly one of four states at any moment, and understanding the transitions is what makes RTOS behaviour predictable rather than mysterious.

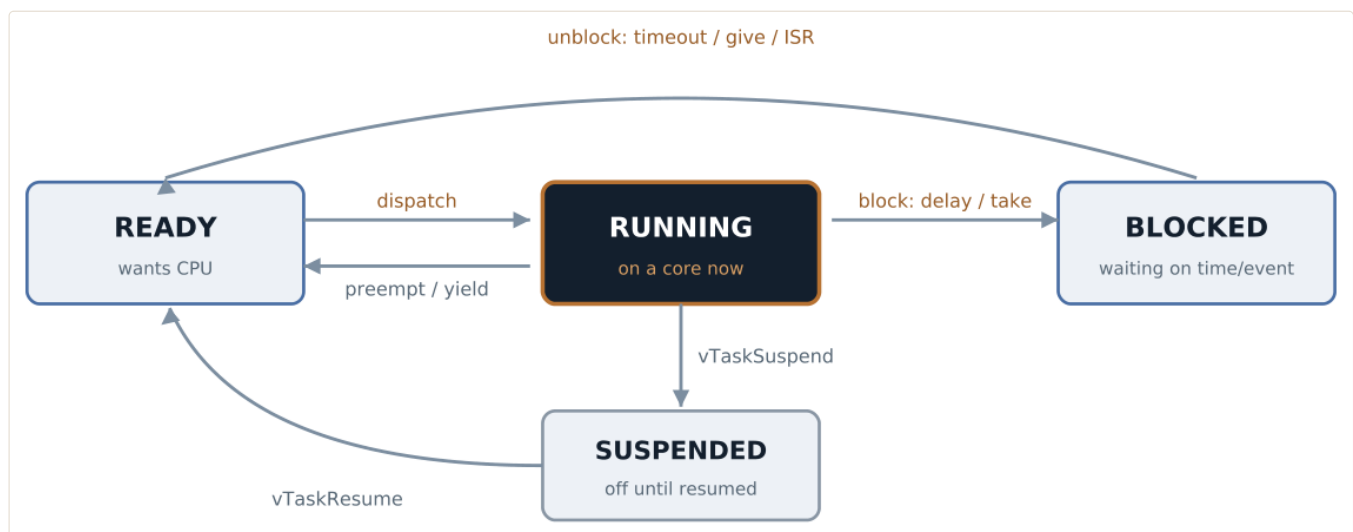


Figure 2.1. Every FreeRTOS task is in exactly one of these states. `vTaskDelay` moves the running task to **BLOCKED (freeing the core); the tick or an event moves it back to **READY**. Your TM4C super-loop had exactly one implicit state, always running, which is why a busy-wait there froze everything.**

FreeRTOS task state diagram. Four states and the transitions between them.

State	Meaning
READY	waiting for CPU
RUNNING	executing on a core right now
BLOCKED	waiting on a timeout or an event
SUSPENDED	explicitly suspended, off the scheduler entirely

Transitions: READY -> RUNNING is "dispatch". RUNNING -> READY is "preempt / yield". RUNNING -> BLOCKED is "block / delay / take". BLOCKED -> READY is "unblock: timeout, give, or ISR". RUNNING -> SUSPENDED is `vTaskSuspend`. SUSPENDED -> READY is `vTaskResume`.

Key point: a BLOCKED task consumes zero CPU. Blocking is the efficient state, not the wasteful one.

RUNNING means actually executing on a core — and because this chip has two cores, two tasks can be running simultaneously. **READY** means it wants the CPU and is waiting its turn. **BLOCKED** is the important one: the task is asleep waiting for time to pass or an event to arrive, and it consumes *no CPU at all* while blocked. **SUSPENDED** is an explicit off switch, used rarely.

The whole value proposition is in the BLOCKED state. A busy-wait keeps a task RUNNING while it accomplishes nothing; a blocking call moves it out of contention entirely so other work proceeds, and the kernel wakes it the instant it is ready.

Drift: why two delay functions exist

This is the subtle part of the lab and a genuine interview question. `vTaskDelay(N)` means "block me for N ticks *starting now*." The catch is that "now" is the moment the call executes, *after* whatever work the loop just did. So the real period becomes (work time + N), and since work time varies, the period varies. Over minutes the errors accumulate and your "1 Hz" heartbeat drifts measurably.

`vTaskDelayUntil(&last, N)` works differently: it holds an **absolute** tick count in `last` and sleeps until `last + N`, then advances `last` by exactly N. The wake-ups land on a fixed grid regardless of how long the work took, so there is no accumulated drift. Any periodic task that must keep real time uses this form.

NOTE

The limit that motivates Lab 4

Both functions are quantised to the tick, which defaults to **100 Hz** (10 ms). So the finest period you can request is 10 ms, and `vTaskDelay(1)` is a 10 ms delay, not 1 ms. That means **you cannot sample at 1 kHz with an RTOS delay**, no matter how you write it. That limitation is exactly why Lab 4 introduces a hardware timer, and why Lab 7's pipeline is driven by `esp_timer` rather than by a task delay. Remember this; it is load-bearing.

The tick is also *not* the same idea as the TM4C's timer prescaler. A prescaler trades resolution for range on one peripheral. The tick is a system-wide scheduling quantum — a boundary you cannot get underneath using task delays at all.

2.4 Procedure & code

Two tasks: a 1 Hz heartbeat using drift-free timing, and a roughly 4 Hz worker using an ordinary delay, so you can watch both interleave in the monitor.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

static const char *TAG = "lab2";

static void heartbeat_task(void *arg) {
    TickType_t last = xTaskGetTickCount();
    while (1) {
        ESP_LOGI(TAG, "heartbeat");
        vTaskDelayUntil(&last, pdMS_TO_TICKS(1000)); // wakes on a fixed 1 Hz grid -- no drift
    }
}

static void worker_task(void *arg) {
    int n = 0;
    while (1) {
        ESP_LOGI(TAG, "worker tick %d", n++);
        vTaskDelay(pdMS_TO_TICKS(250)); // ~4 Hz, drifts by however long the log took
    }
}

void app_main(void) {
    xTaskCreate(heartbeat_task, "hb", 2048, NULL, 5, NULL); // name, stack bytes, arg, priority,
    handle
    xTaskCreate(worker_task, "wk", 2048, NULL, 5, NULL);
    // app_main can return here -- the two tasks keep running under the scheduler.
}
```

2.5 Code walkthrough, line by line

Twenty-seven lines, and the last three are the punchline.

Lines 1-3 · the three includes

`FreeRTOS.h` must come first — it defines the core types (including `TickType_t` on line 8) that the others rely on. `task.h` brings the task calls. `esp_log.h` is ESP-IDF's logging system, home of the `ESP_LOGI` macro.

Line 5 · `static const char *TAG = "lab2";`

A **tag** is a short string identifying which subsystem a log line came from. ESP-IDF prints it in every message and can filter by it at runtime, which matters once several modules are logging at once. The idiom is one `TAG` per `.c` file.

Read the declaration right to left: `TAG` is a *pointer* to `char`, the characters are `const` (they must not be modified), and the whole thing is `static` (private to this file). **TM4C bridge:** you had `printf` over UART with no levels and no filtering. This is the professional version of the same thing.

Line 7 · `static void heartbeat_task(void *arg)`

Every FreeRTOS task function has this exact shape: returns `void`, takes one `void *`. **`void *` is a "pointer to anything"** — C's way of passing an argument whose type the kernel cannot possibly know, so you cast it back to whatever you actually passed. Here nothing is passed (line 24 passes `NULL`), so `arg` is unused.

Line 8 · `TickType_t last = xTaskGetTickCount();`

`TickType_t` is FreeRTOS's integer type for a tick count — a typedef, so its actual width depends on configuration. `xTaskGetTickCount()` returns ticks elapsed since boot, which seeds the anchor that line 11 will advance. Note the `x` prefix: FreeRTOS convention for a function that returns a value.

This line is *outside* the `while` deliberately. It establishes the starting point once; putting it inside would reset the anchor every pass and destroy the whole drift-free property.

Line 10 · `ESP_LOGI(TAG, "heartbeat");`

`ESP_LOGI` is the **Info** level. The family is `ESP_LOGE` (error), `ESP_LOGW` (warn), `ESP_LOGI`, `ESP_LOGD` (debug) and `ESP_LOGV` (verbose), and you can set the visible level at compile time or runtime. Lab 16 uses three of them at once to separate interleaved output; it is worth getting into the habit of choosing the right one rather than defaulting to `I`.

Line 11 · `vTaskDelayUntil(&last, pdMS_TO_TICKS(1000));`

The technically interesting line in the lab. It sleeps until the absolute tick `last + 1000 ms`, then advances `last` by exactly that amount. Because the target is absolute rather than relative, however long line 10 took is irrelevant — wake-ups land on a fixed grid and errors never accumulate.

The `&` is essential and is the thing people get wrong. The function must *update* your variable to the next wake time, and C passes by value, so it needs the address. Pass `last` instead of `&last` and it will not compile — which is fortunate, because a version that silently did the wrong thing would be far worse.

Line 16 · `int n = 0;`

Declared outside the `while` on line 17 so it survives across iterations. Inside, it would reset to 0 every pass and the counter would never climb — the same scoping point as `bool on` in Lab 1.

Line 18 · `ESP_LOGI(TAG, "worker tick %d", n++);`

`n++` is **post-increment**: the current value is passed to the log call, then `n` is incremented. So the first line prints 0, not 1. Pre-increment (`++n`) would print 1 first. This distinction shows up again in Lab 7, where `win_buf[win_idx++]` depends on exactly this behaviour.

Line 19 · `vTaskDelay(pdMS_TO_TICKS(250));`

The *relative* delay, deliberately contrasted with line 11. It sleeps 250 ms starting from **now** — after the log call has already taken some time — so the true period is (work + 250 ms) and it drifts. For a status

message nobody is timing, that is completely fine. The lab runs both side by side so you can see that "which delay do I use" is a real decision, not a style preference.

Lines 24-25 · `xTaskCreate(heartbeat_task, "hb", 2048, NULL, 5, NULL);`

Six arguments, in order:

#	Argument	Meaning
1	<code>heartbeat_task</code>	The function to run. Passing the bare name passes its address.
2	<code>"hb"</code>	A short name, used in crash dumps and debugger task lists.
3	<code>2048</code>	Stack size. In ESP-IDF this is bytes ; upstream FreeRTOS counts <i>words</i> . A genuine portability trap.
4	<code>NULL</code>	The argument handed to the task as its <code>void *arg</code> . Nothing needed here.
5	<code>5</code>	Priority. Higher numbers win. Both tasks get 5, so they are peers and the kernel time-slices.
6	<code>NULL</code>	Where to store a handle to the task. <code>NULL</code> means "I will never need to refer to it again."

NOTE

Why 2048, and how you would actually know

It is a reasonable default for a task that only logs. Too small gives a stack-overflow crash; too large wastes RAM you do not have much of. The professional answer is that you *measure* with `uxTaskGetStackHighWaterMark()`, which reports the least free stack a task has ever had, then size with margin. Per-task stack sizing is a named gap on most junior résumés and §2.7 closes it in ten minutes.

Lines 26-27 · `// app_main can return here`

The comment is the point of the whole lab. `app_main` spawns two tasks and then **returns**, and the program keeps running, because the tasks are independent entities owned by the scheduler rather than things called from here. **TM4C bridge:** on the TM4C, `main()` returning would have been catastrophic — it *was* the program. Here it is a launcher that is allowed to finish.

2.6 Docs & references

DOCS

Trace every line to its source

`xTaskCreate`, `vTaskDelay`, `vTaskDelayUntil`, `xTaskGetTickCount`, `uxTaskGetStackHighWaterMark` — FreeRTOS API reference, task creation and control. **Check the stack-size units yourself** in the ESP-IDF FreeRTOS page; this is the one place the ESP port differs from upstream in a way that bites.

`ESP_LOGI` and log levels — ESP-IDF API Reference → System → **Logging**. Look at how to change the level at runtime and per tag.

Tick rate (`CONFIG_FREERTOS_HZ`) — `menuconfig` → Component config → FreeRTOS → Kernel. Confirm it says 100.

ESP-IDF FreeRTOS differences — Programming Guide → API Reference → System → **FreeRTOS (Overview)**. This documents SMP behaviour and core pinning, which Lab 7 relies on.

2.7 Verify, troubleshoot, your turn

Verify. The monitor shows "heartbeat" once a second and "worker tick N" about four times a second, interleaved. Two independent flows of execution from one program, with no super-loop coordinating them — that interleaving *is* the result.

Troubleshoot

Symptom	Cause and fix
Only one task's output appears	The other task crashed at startup, likely a stack overflow. Raise its stack and read the boot log carefully.
Guru Meditation right after start	A task function returned instead of looping forever, or <code>xTaskCreate</code> was given a bad function pointer.
<code>vTaskDelayUntil</code> compile error	Check the <code>&</code> on <code>last</code> . It takes a pointer, not a value.
Timing is coarser than requested	Expected. Everything quantises to the 10 ms tick. This is the Lab 4 motivation, not a bug.

Your turn

Three experiments, in order of how much they teach.

1. Give the worker priority 10 and the heartbeat 5, then put a deliberate busy-wait (a counting loop, *not* `vTaskDelay`) inside the worker. Watch the heartbeat starve. You have just reproduced CPU starvation on purpose, and you now understand why blocking calls matter.
2. Add a variable-length delay before the `vTaskDelayUntil` in the heartbeat and confirm the 1 Hz grid holds anyway. Do the same with `vTaskDelay` and watch it drift.
3. Call `uxTaskGetStackHighWaterMark(NULL)` from inside each task and log it. Now you know how much of your 2048 bytes you are really using, and you can size the next task from evidence instead of habit.

Log all three in `debug_log.md`. The starvation experiment in particular is a genuine interview story.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 2.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02 ←	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.